

Shor's Algorithm for the Elliptic Curve Discrete Logarithm Problem

Complete algorithm, full derivations, and circuits

August 2, 2026

Abstract

This is a self-contained, derivation-first companion on solving the elliptic curve discrete logarithm problem (ECDLP) with Shor's algorithm. We build the whole tower from the top down: the ECDLP is an instance of the abelian hidden subgroup problem, so it reduces to *two-dimensional* phase estimation; the oracle is elliptic-curve scalar multiplication; scalar multiplication is a cascade of controlled point additions; a point addition is a fixed sequence of modular field operations; and each field operation – addition, doubling, multiplication, squaring, inversion – is a reversible circuit over Toffoli gates. Every step is derived, not asserted: the group law from the chord–tangent construction, the measured linear relation $y_2 \equiv m y_1$ from character orthogonality, the MAJ/UMA adder from a one-line Boolean identity, and Kaliski's binary-GCD inversion made reversible by a round counter. Complete circuits are given for the full and semiclassical algorithms, for every arithmetic block, and for the controlled point addition. A toy curve over $\mathbb{F}_5$ is carried end to end, and the section closes with the Roetteler–Naehrig–Svore–Lauter resource estimates and the comparison that makes elliptic curves an easier quantum target than RSA.

The reduction tower (read top to bottom)

$$\begin{array}{c} \text{ECDLP} \xrightarrow{\text{HSP}} \text{2D phase estimation} \xrightarrow{\text{oracle}} \text{scalar mult } [k]P + [\ell]Q \\ \xrightarrow{\text{double-and-add}} \text{controlled point addition} \xrightarrow{\text{group law}} \text{modular field ops} \xrightarrow{\text{reversible}} \text{Toffoli circuits.} \end{array}$$

Section 1 derives the top arrow; section 2 the group law; section 3 the algorithm and its two circuits; section 4 the field operations; section 5 the point addition; section 6 a full example; section 7 the costs.

Contents

1	The discrete logarithm as a hidden subgroup problem	2
1.1	Setup and notation	2
1.2	The hiding function and its hidden subgroup	3
1.3	State evolution and the measured relation (exact case)	3
1.4	The finite-register version and post-processing	4
1.5	Multiplicative vs elliptic-curve discrete logarithm	4
2	Elliptic curves over a prime field	4
2.1	The curve and its group	5
2.2	Deriving the group law	5
2.3	Scalar multiplication, order, and register size	6
2.4	The running curve	6

3	The quantum ECDLP algorithm and its circuits	6
3.1	The algorithm	6
3.2	The full (parallel) circuit	7
3.3	The semiclassical circuit (one extra qubit)	7
3.4	Scalar multiplication is a cascade of conditional point additions	8
4	Reversible modular arithmetic	8
4.1	Integer addition: the MAJ/UMA ripple-carry adder	8
4.2	Modular addition	9
4.3	Modular doubling	9
4.4	Modular multiplication and squaring	10
4.5	Modular inversion (the expensive block)	10
5	Reversible controlled point addition	10
5.1	The operation sequence	11
5.2	Exceptional cases	11
6	Worked example over $\mathbb{F}_5$	12
7	Resources and comparison with RSA	12
7.1	Asymptotic and concrete counts	13
7.2	Elliptic curves vs. RSA	13
8	Summary	13
A	Gate and cost reference	14

1 The discrete logarithm as a hidden subgroup problem

What this section establishes

The ECDLP is solved by the *same* eigenvalue-to-phase mechanism as factoring, but on a *two*-dimensional hidden subgroup of $\mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$. We derive, from character orthogonality alone, that measuring the two index registers returns a pair (y_1, y_2) obeying the linear relation $y_2 \equiv m y_1 \pmod{r}$, from which m is read off by one modular inversion.

1.1 Setup and notation

Let $G = \langle g \rangle$ be a finite cyclic group of known order r , written additively for the elliptic-curve case. Fix a generator and an element

$$h = [m]g \quad (\text{multiplicatively: } h = g^m),$$

for an unknown $m \in \mathbb{Z}/r\mathbb{Z}$. The **discrete logarithm problem** is to recover m . For the ECDLP, $G = \langle P \rangle$ is the cyclic subgroup generated by a point P of prime order r on an elliptic curve $E(\mathbb{F}_p)$, and $Q = [m]P$; find m . Throughout, $[k]P$ denotes the k -fold sum $P + \cdots + P$ (section 2), $\omega_r = e^{2\pi i/r}$, and $\text{QFT}_r|x\rangle = \frac{1}{\sqrt{r}} \sum_y \omega_r^{xy}|y\rangle$.

Definition 1.1 (Hidden subgroup problem). Given a function $f : A \rightarrow X$ on a finite abelian group A that is *constant on cosets* of an unknown subgroup $L \leq A$ and *distinct on distinct cosets*, find L . We say f *hides* L .

1.2 The hiding function and its hidden subgroup

Take $A = \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ and define

$$f(k, \ell) = [k]P + [\ell]Q.$$

Because $Q = [m]P$ and scalar multiplication is a group homomorphism $\mathbb{Z} \rightarrow G$,

$$f(k, \ell) = [k]P + [\ell][m]P = [k]P + [\ell m]P = [k + m\ell]P,$$

so $f(k, \ell)$ depends only on $k + m\ell \bmod r$.

Proposition 1.2 (The hidden subgroup). *f hides the cyclic subgroup*

$$L = \{(k, \ell) \in A : k + m\ell \equiv 0 \pmod{r}\} = \langle (-m, 1) \rangle,$$

of order r and index $[A : L] = r$. Knowing L is equivalent to knowing m .

Proof. $f(k, \ell) = f(k', \ell') \iff k + m\ell \equiv k' + m\ell' \iff (k - k') + m(\ell - \ell') \equiv 0 \iff (k - k', \ell - \ell') \in L$, so f is constant on cosets of L and injective across them. Every element of L has the form $(-m\ell, \ell) = \ell \cdot (-m, 1)$, so $L = \langle (-m, 1) \rangle$; it has r elements, whence index r . Reading m off the generator $(-m, 1)$ recovers m . $\square$

1.3 State evolution and the measured relation (exact case)

Assume for the derivation that the index registers have dimension exactly r (the 2^{n+1} -dimensional case is section 1.4). The algorithm is: create the uniform superposition over A , apply f into a point register, then QFT_r to each index register, and measure.

Theorem 1.3 (The linear relation). *Running the above yields a measurement outcome $(y_1, y_2) \in \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ with*

$$y_2 \equiv m y_1 \pmod{r},$$

each admissible y_1 occurring with probability $1/r$.

Proof. (1) *Superposition and oracle.*

$$\frac{1}{r} \sum_{k, \ell \in \mathbb{Z}/r\mathbb{Z}} |k\rangle|\ell\rangle \xrightarrow{f} \frac{1}{r} \sum_{k, \ell} |k\rangle|\ell\rangle |[k + m\ell]P\rangle.$$

(2) *Measure (or, by deferred measurement, ignore) the point register.* It holds $|[c]P\rangle$ for a uniformly random coset label $c \in \mathbb{Z}/r\mathbb{Z}$; the index registers collapse to the uniform superposition over the coset $\{(k, \ell) : k + m\ell \equiv c\}$. Parametrising that coset by the free variable ℓ (so $k = c - m\ell$),

$$\frac{1}{\sqrt{r}} \sum_{\ell \in \mathbb{Z}/r\mathbb{Z}} |c - m\ell\rangle|\ell\rangle.$$

(3) *Apply $\text{QFT}_r \otimes \text{QFT}_r$:*

$$\begin{aligned} & \frac{1}{\sqrt{r}} \sum_{\ell} \left(\frac{1}{\sqrt{r}} \sum_{y_1} \omega_r^{(c - m\ell)y_1} |y_1\rangle \right) \left(\frac{1}{\sqrt{r}} \sum_{y_2} \omega_r^{\ell y_2} |y_2\rangle \right) \\ &= \frac{1}{r^{3/2}} \sum_{y_1, y_2} \omega_r^{c y_1} \underbrace{\left(\sum_{\ell} \omega_r^{\ell(y_2 - m y_1)} \right)}_{= r [y_2 \equiv m y_1]} |y_1\rangle |y_2\rangle = \frac{1}{\sqrt{r}} \sum_{y_1} \omega_r^{c y_1} |y_1\rangle |m y_1 \bmod r\rangle, \end{aligned}$$

using the orthogonality $\sum_{\ell=0}^{r-1} \omega_r^{\ell t} = r [t \equiv 0 \bmod r]$. (4) *Measure.* The global phase $\omega_r^{c y_1}$ is unobservable; the outcome is $(y_1, m y_1 \bmod r)$, uniform over the r values of y_1 . $\square$

Corollary 1.4 (Recovering m). *If $\gcd(y_1, r) = 1$ then $m \equiv y_2 y_1^{-1} \pmod{r}$, computed by one extended-Euclid inversion. When r is **prime** (the NIST curves, cofactor 1), every $y_1 \neq 0$ is invertible, so a single run succeeds with probability $(r-1)/r \approx 1$.*

Proof. $y_2 \equiv m y_1$ and y_1 invertible give $m \equiv y_2 y_1^{-1}$. For prime r , $y_1 \in \{1, \dots, r-1\}$ is invertible; only $y_1 = 0$ (probability $1/r$) is uninformative. $\square$

Why two QFTs, and why it is the same mechanism as factoring

The translations $(k, \ell) \mapsto (k+a, \ell+b)$ of $A = \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ commute, so they are simultaneously diagonalised by the characters $\chi_{y_1, y_2}(k, \ell) = \omega_r^{ky_1 + \ell y_2}$; $\text{QFT}_r \otimes \text{QFT}_r$ is the change to that eigenbasis, and it factors over the two coordinates because $\widehat{A} \cong \widehat{\mathbb{Z}/r\mathbb{Z}} \times \widehat{\mathbb{Z}/r\mathbb{Z}}$. Measuring samples a character that is *trivial on the hidden subgroup* L : the condition $\chi_{y_1, y_2}|_L \equiv 1$ is exactly $y_2 \equiv m y_1$. Factoring is the one-dimensional case ($A = \mathbb{Z}/r\mathbb{Z}$, $L = \langle r' \rangle$); here A is two-dimensional and one measured character pins down the line L , hence m .

1.4 The finite-register version and post-processing

In practice r is unknown in advance and not a power of two, so each index register uses $n+1$ qubits with $2^{n+1} > r$ (possible because the Hasse bound gives $r \leq \#E(\mathbb{F}_p) \leq p+1+2\sqrt{p} < 2^{n+1}$ for $2^n > p$), and one applies $\text{QFT}_{2^{n+1}}$. The exact delta function of theorem 1.3 broadens into a Fejér peak (as in ordinary phase estimation), and the measured pair satisfies

$$\frac{y_1}{2^{n+1}} \approx \frac{j_1}{r}, \quad \frac{y_2}{2^{n+1}} \approx \frac{j_2}{r}, \quad j_2 \equiv m j_1 \pmod{r},$$

for integers j_1, j_2 . Rounding $(\frac{y_1}{2^{n+1}}, \frac{y_2}{2^{n+1}})$ to the nearest rationals with denominator r (equivalently, a two-dimensional continued-fraction / lattice step) recovers (j_1, j_2) and hence $m \equiv j_2 j_1^{-1}$; a handful of runs make this reliable. We treat the exact case as the conceptual core and refer to the standard analysis for the error bounds.

1.5 Multiplicative vs elliptic-curve discrete logarithm

The derivation used only that G is cyclic of order r and that the oracle computes the group operation. The finite-field DLP ($h = g^m$ in $\mathbb{F}_p^\times$, $r = p-1$) and the ECDLP ($Q = [m]P$ in $E(\mathbb{F}_p)$) are therefore the identical algorithm with two different oracles:

setting	oracle $f(k, \ell)$	cost of the oracle
finite-field DLP	$g^k h^\ell \pmod{p}$	modular exponentiation
ECDLP	$[k]P + [\ell]Q$	elliptic scalar multiplication

The rest of this document builds the elliptic-curve oracle and its circuit; that oracle is where all the engineering – and 99% of the gates – lives.

2 Elliptic curves over a prime field

What this section establishes

The oracle needs the group law $+$ on $E(\mathbb{F}_p)$ in explicit coordinate formulas, because those formulas are what the reversible circuit must implement. We derive them from the chord–tangent construction and read off exactly which field operations appear – crucially, a modular *inversion*.

2.1 The curve and its group

Let $p > 3$ be prime and $\mathbb{F}_p$ the field with p elements. An **elliptic curve** in short Weierstrass form is

$$E : y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p, \quad \Delta = -16(4a^3 + 27b^2) \neq 0,$$

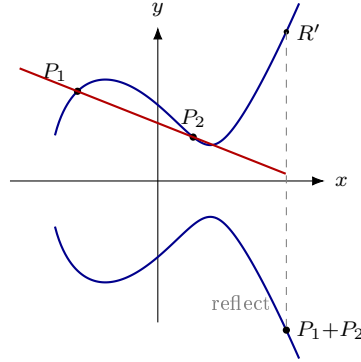
the non-vanishing discriminant ensuring no repeated roots (a smooth curve). Its set of points is

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

where $\mathcal{O}$ is the *point at infinity*. Under the chord–tangent addition below, $E(\mathbb{F}_p)$ is a finite abelian group with identity $\mathcal{O}$ and inverse $-(x, y) = (x, -y)$.

2.2 Deriving the group law

Three points of E on a common line sum to $\mathcal{O}$; this defines $+$. Concretely, to add $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ with $P_1 \neq \pm P_2$, draw the line through them and reflect its third intersection with E .



Theorem 2.1 (Affine addition formulas). For $P_1 \neq \pm P_2$ put $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$; for doubling ($P_1 = P_2, y_1 \neq 0$) put $\lambda = \frac{3x_1^2 + a}{2y_1}$. Then $P_3 = P_1 + P_2 = (x_3, y_3)$ with

$$\boxed{x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.}$$

Proof. The line through P_1, P_2 is $y = \lambda x + \nu$ with $\nu = y_1 - \lambda x_1$. Substituting into $y^2 = x^3 + ax + b$,

$$(\lambda x + \nu)^2 = x^3 + ax + b \implies x^3 - \lambda^2 x^2 + (a - 2\lambda\nu)x + (b - \nu^2) = 0.$$

This cubic has roots x_1, x_2, x_3 (the three intersection points), so by Viète the sum of roots equals the negated x^2 -coefficient: $x_1 + x_2 + x_3 = \lambda^2$, giving $x_3 = \lambda^2 - x_1 - x_2$. The third intersection is $(x_3, \lambda x_3 + \nu)$; reflecting in the x -axis (the group inverse) gives $y_3 = -(\lambda x_3 + \nu) = -(\lambda x_3 + y_1 - \lambda x_1) = \lambda(x_1 - x_3) - y_1$. For doubling, differentiate $y^2 = x^3 + ax + b$ implicitly: $2yy' = 3x^2 + a$, so the tangent slope is $\lambda = (3x_1^2 + a)/(2y_1)$, and the same Viète argument (with a double root at x_1) gives the formulas. $\square$

The operation that dominates the cost

Both slopes are a *quotient* in $\mathbb{F}_p$: computing λ requires a modular **inversion** $(x_2 - x_1)^{-1}$ or $(2y_1)^{-1}$. Modular inversion (section 4.5) is by far the most expensive field operation, and it is the reason an elliptic-curve point addition is much dearer than a modular multiplication – and why 256-bit ECDLP rivals 2048-bit RSA in quantum cost despite the smaller key.

2.3 Scalar multiplication, order, and register size

For $m \in \mathbb{Z}$, $[m]P = \underbrace{P + \dots + P}_m$ ($[0]P = \mathcal{O}$, $[-m]P = [m](-P)$) is a group homomorphism $\mathbb{Z} \rightarrow E(\mathbb{F}_p)$; its image $\langle P \rangle$ is cyclic of order $r = \text{ord}(P)$ (least $r \geq 1$ with $[r]P = \mathcal{O}$). Scalar multiplication is computed by **double-and-add**: writing $m = \sum_j m_j 2^j$,

$$[m]P = \sum_j m_j [2^j]P, \quad [2^j]P \text{ by } j \text{ repeated doublings,}$$

which is the classical analogue of repeated squaring and the structure the quantum circuit mirrors (section 3). By Hasse's theorem $|\#E(\mathbb{F}_p) - (p+1)| \leq 2\sqrt{p}$, so $r \leq \#E < p+1+2\sqrt{p} < 2^{n+1}$ where $n = \lceil \log_2 p \rceil$; hence $n+1$ **qubits** suffice to hold any scalar or coordinate.

2.4 The running curve

We fix a toy curve, used again in section 6:

$$E: y^2 = x^3 + 3x + 2 \quad \text{over } \mathbb{F}_5.$$

Testing each x (squares in $\mathbb{F}_5$ are $\{0, 1, 4\}$): $x=1 \rightarrow 6 \equiv 1$ (a square, $y = \pm 1$); $x=2 \rightarrow 16 \equiv 1$ ($y = \pm 1$); $x=0, 3, 4$ give non-squares. Hence

$$E(\mathbb{F}_5) = \{\mathcal{O}, (1, 1), (1, 4), (2, 1), (2, 4)\}, \quad \#E(\mathbb{F}_5) = 5 \text{ (prime),}$$

so every non-identity point generates. Take $P = (2, 1)$. Doubling ($a = 3$): $\lambda = \frac{3 \cdot 4 + 3}{2} = \frac{0}{2} = 0$, $x = 0 - 4 \equiv 1$, $y = 0 - 1 \equiv 4$, so $[2]P = (1, 4)$. Continuing with theorem 2.1:

$$[1]P = (2, 1), \quad [2]P = (1, 4), \quad [3]P = (1, 1), \quad [4]P = (2, 4), \quad [5]P = \mathcal{O}.$$

Thus for $Q = (1, 1)$ the discrete log is $\boxed{m=3}$ ($Q = [3]P$) – the answer the quantum algorithm will produce without knowing it in advance.

3 The quantum ECDLP algorithm and its circuits

What this section establishes

We turn theorem 1.3 into an explicit circuit: two index registers, a point register, a Hadamard layer, a *double scalar multiplication* built from controlled point additions of the fixed points $[2^j]P$ and $[2^j]Q$, two quantum Fourier transforms, and measurement. We give both the parallel circuit and the semiclassical single-ancilla circuit, and derive the latter's feedback phases.

3.1 The algorithm

Algorithm 3.1 (Shor for the ECDLP). Input: $E(\mathbb{F}_p)$, a point P of order r , and $Q = [m]P$; set $n = \lceil \log_2 p \rceil$.

1. Prepare two index registers $|0\rangle_{n+1}|0\rangle_{n+1}$ and a point register in $|\mathcal{O}\rangle$.
2. Apply $H^{\otimes(n+1)}$ to each index register: $\frac{1}{2^{n+1}} \sum_{k,\ell} |k\rangle|\ell\rangle|\mathcal{O}\rangle$.
3. **Double scalar multiplication** (the oracle): controlled on bit k_j add the classical point $[2^j]P$; controlled on bit ℓ_j add $[2^j]Q$. This maps the point register to $[k]P + [\ell]Q$:

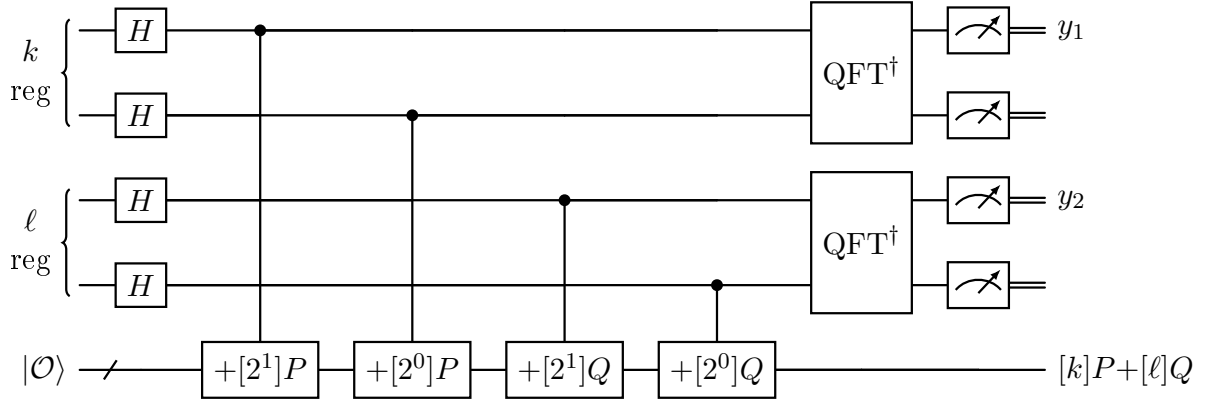
$$\frac{1}{2^{n+1}} \sum_{k,\ell} |k\rangle|\ell\rangle|[k]P + [\ell]Q\rangle.$$

4. Apply $\text{QFT}_{2^{n+1}}^\dagger$ to each index register.
5. Measure both index registers, obtaining (y_1, y_2) .
6. Classically recover m : rescale/round to (j_1, j_2) with $j_2 \equiv mj_1 \pmod{r}$ and output $m \equiv j_2 j_1^{-1} \pmod{r}$ (corollary 1.4). For prime r , one run with $y_1 \neq 0$ suffices; otherwise repeat a few times.

The point register is discarded after step 4; only the $2(n+1)$ index bits are measured.

3.2 The full (parallel) circuit

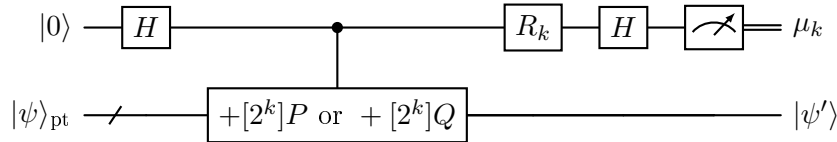
Each fixed point $[2^j]P, [2^j]Q$ is precomputed *classically*, so every box below is a controlled addition of a *constant* point (section 5).



(Shown with $n+1 = 2$ bits per register; in general the k -register contributes the ladder $+ [2^n]P, \dots, + [2^0]P$ and the ℓ -register $+ [2^n]Q, \dots, + [2^0]Q$.) Additions commute, so their order is free. The two $\text{QFT}^\dagger$ blocks are the change to the character basis of $\mathbb{Z}/2^{n+1}\mathbb{Z} \times \mathbb{Z}/2^{n+1}\mathbb{Z}$; measurement samples the relation of theorem 1.3.

3.3 The semiclassical circuit (one extra qubit)

Every control of the two $\text{QFT}^\dagger$ s is a qubit measured at the end and never reused, so by deferred measurement each two-qubit rotation of the inverse QFT can be replaced by a *classically controlled* one-qubit rotation applied after measuring its control (the Griffiths–Niu semiclassical QFT). This collapses each index register to a *single* reused ancilla: process bits from most to least significant, feeding back a phase correction.



Proposition 3.2 (Feedback phase). *Processing bit index $k = n, n-1, \dots, 0$ of a register, the correction gate is $R_k = \text{diag}(1, e^{i\theta_k})$ with*

$$\theta_k = -\pi \sum_{j>k} 2^{k-j} \mu_j,$$

where $\mu_j \in \{0, 1\}$ are the already-measured higher bits. With this feedback the measured bits $\mu_n \dots \mu_0$ equal the outcome that the parallel $\text{QFT}^\dagger$ would have produced, using *one* ancilla qubit besides the point register.

Sketch. The inverse QFT on bit k is a Hadamard preceded by controlled phases $\text{diag}(1, e^{-2\pi i 2^{k-j}})$ from each higher bit j . After those higher bits are measured to values μ_j , the net phase on the $|1\rangle$ branch of bit k is $\exp(-2\pi i \sum_{j>k} \mu_j 2^{k-j-1}) = e^{i\theta_k}$ with $\theta_k = -\pi \sum_{j>k} \mu_j 2^{k-j}$; applying R_k reproduces it, and H then reads bit k . This is the standard semiclassical QFT; it is exact here because 2^{n+1} divides the register dimension. $\square$

The point register never leaves the circuit between steps, so the two index registers are folded into repeated use of the same wire; only one ancilla is needed beyond the register that holds and adds points – the form used in the resource estimates of section 7.

3.4 Scalar multiplication is a cascade of conditional point additions

Proposition 3.3 (Double scalar multiplication). *For a classically known point R and an $(n+1)$ -qubit scalar register $|s\rangle$,*

$$\prod_{j=0}^n \left(\text{controlled-}(+[2^j]R) \text{ on qubit } s_j \right) : |s\rangle|\mathcal{O}\rangle \mapsto |s\rangle|[s]R\rangle,$$

since $[s]R = \sum_j s_j [2^j]R$ and the constants $[2^j]R = [2]([2^{j-1}]R)$ are computed by j classical doublings. Running this for $R = P$ on the k -register and $R = Q$ on the ℓ -register realises step 3 of algorithm 3.1.

Why the additions are by a *constant* point

Because $[2^j]P$ and $[2^j]Q$ are classical constants, each box is a *controlled point addition of a fixed point to a quantum point* – not a general point–point addition. This is materially cheaper: the constant’s coordinates are folded into the gate pattern at compile time, and (unlike RSA) no quantum–quantum multiplication is needed. The one irreducible cost inside each box is the modular inversion for the slope λ (section 5).

4 Reversible modular arithmetic

What this section establishes

A point addition is a fixed sequence of operations in $\mathbb{F}_p$: add, subtract, double, multiply, square, and invert, all modulo p . Each must be a *reversible* circuit (garbage would record which-path information and destroy the interference of theorem 1.3). We build them bottom-up, deriving each from the one below it, and end with the costly inversion.

All registers hold n -bit integers with $p < 2^n$; “ADD” etc. are in-place unless stated. Every constant $(p, 2^i a \bmod p, \dots)$ is folded in at compile time, which roughly halves the gates (a constant control deletes or fixes a Toffoli).

4.1 Integer addition: the MAJ/UMA ripple-carry adder

Schoolbook binary addition is $s_i = a_i \oplus b_i \oplus c_i$, $c_{i+1} = \text{MAJ}(a_i, b_i, c_i) = a_i b_i \oplus a_i c_i \oplus b_i c_i$, $c_0 = 0$. The sum bits are $\mathbb{F}_2$ -linear (free CNOTs); the carries are the only nonlinear part, so *an adder’s cost is the cost of its carries*.

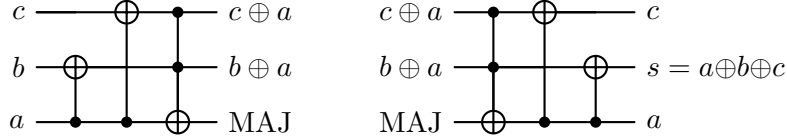
Lemma 4.1 (MAJ and UMA blocks). *On an ordered triple (c, b, a) define, in order,*

$$\text{MAJ} : \text{CNOT}(a \rightarrow b), \text{CNOT}(a \rightarrow c), \text{TOF}(c, b \rightarrow a),$$

$$\text{UMA} : \text{TOF}(c, b \rightarrow a), \text{CNOT}(a \rightarrow c), \text{CNOT}(c \rightarrow b).$$

Then MAJ sends $(c, b, a) \mapsto (c \oplus a, b \oplus a, \text{MAJ}(a, b, c))$, writing the outgoing carry onto the a -wire; UMA is its partial inverse, restoring a, c and leaving the sum bit $a \oplus b \oplus c$ on the b -wire. Each block uses exactly one Toffoli.

Proof. Track MAJ: $b \mapsto b \oplus a$, $c \mapsto c \oplus a$, then the Toffoli sends $a \mapsto a \oplus (c \oplus a)(b \oplus a) = a \oplus (cb \oplus ca \oplus ab \oplus a) = ab \oplus ac \oplus bc$ (using $a \oplus a = 0$), which is $\text{MAJ}(a, b, c)$. Feeding this into UMA: the Toffoli restores a , the $\text{CNOT}(a \rightarrow c)$ restores c , and $\text{CNOT}(c \rightarrow b)$ turns $b \oplus a$ into $a \oplus b \oplus c$, the sum bit. $\square$



Theorem 4.2 (CDKM adder). *Interleaving wires $c_0, b_0, a_0, b_1, a_1, \dots, b_{n-1}, a_{n-1}, z$ and running a MAJ chain up, one CNOT copying the top carry to z , then a UMA chain down computes*

$$|c_0=0\rangle|b\rangle|a\rangle|z=0\rangle \mapsto |0\rangle|(a+b) \bmod 2^n\rangle|a\rangle|s_n\rangle,$$

*i.e. an in-place $(n+1)$ -bit sum with a preserved and **no garbage**, using $2n$ Toffoli, $2n+2$ qubits. Dropping z gives a mod- 2^n adder on $2n+1$ qubits; reversing it subtracts. (Roetteler et al. use the equivalent Takahashi adder.)*

The Gidney *temporary-AND* trick lowers each Toffoli in a compute/uncompute pair to $4T$ gates, so the adder costs $\approx 4nT$.

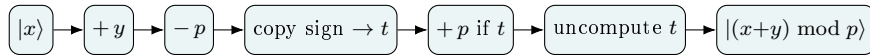
4.2 Modular addition

Proposition 4.3 (Modular adder). *For $0 \leq x, y < p$, the circuit*

$$\text{ADD} \rightarrow \text{SUB}(p) \rightarrow [\text{sign} \rightarrow t] \rightarrow \text{ADD}(p) \text{ ctrl } t \rightarrow [\text{uncompute } t]$$

maps $|x\rangle|y\rangle \mapsto |x\rangle|(x+y) \bmod p\rangle$ using one integer addition, a conditional constant addition of p , one ancilla t , and $\Theta(n)$ Toffoli.

Proof. ADD gives $x + y \in [0, 2p)$; subtracting p gives $x + y - p \in [-p, p)$, whose sign bit is 1 exactly when $x + y < p$. Copy that bit to t ; adding p back conditioned on t yields $(x + y) \bmod p \in [0, p)$. Finally t is recomputable from the output: a reduction occurred (no add-back, $t = 0$) iff $(x + y) \bmod p < x$, i.e. t equals the sign of result $-x$, so one comparison clears the ancilla. $\square$



4.3 Modular doubling

Proposition 4.4 (Modular doubling, p odd). $|x\rangle \mapsto |2x \bmod p\rangle$ is a left bit-shift (multiply by 2) followed by a conditional subtraction of p : shift, $\text{SUB}(p)$, and if the result is negative add p back. The reduction flag is recoverable from the least significant bit ($2x - p$ is odd since p is odd, $2x$ even), so no persistent ancilla is needed; $n+2$ qubits, $\Theta(n)$ Toffoli.

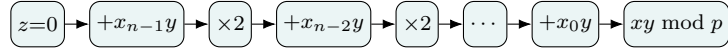
4.4 Modular multiplication and squaring

Theorem 4.5 (Modular multiplier, double-and-add). *By Horner’s rule,*

$$x \cdot y = \sum_{i=0}^{n-1} x_i 2^i y = \left(\cdots ((x_{n-1}y) \cdot 2 + x_{n-2}y) \cdot 2 + \cdots \right) \cdot 2 + x_0 y,$$

so with an accumulator z (start 0), looping $i = n-1, \dots, 0$ with

$z \leftarrow 2z \bmod p$ (proposition 4.4), $z \leftarrow (z + y) \bmod p$ controlled on x_i (proposition 4.3), computes $|x\rangle|y\rangle|0\rangle \mapsto |x\rangle|y\rangle|xy \bmod p\rangle$ in n modular doublings and n controlled modular additions, $\Theta(n^2)$ Toffoli, $3n+2$ qubits. **Squaring** $|x\rangle|0\rangle \mapsto |x\rangle|x^2 \bmod p\rangle$ saves the y -register by copying bit x_i into one ancilla each round ($2n+3$ qubits).



(all $+$ and $\times 2$ are modulo p ; each $+x_i y$ is controlled on x_i .)

4.5 Modular inversion (the expensive block)

The slope λ needs $x^{-1} \bmod p$. This is computed by a *reversible* version of Kaliski’s binary extended-GCD (“almost inverse”) algorithm.

Algorithm 4.6 (Kaliski binary GCD, functional form). Maintain (u, v, r, s) with the loop invariant $p = rv + su$ (Montgomery domain). Start $u = p$, $v = x$, $r = 0$, $s = 1$ and iterate:

$$\begin{cases} u \text{ even} : & u \leftarrow u/2, s \leftarrow 2s; \\ v \text{ even} : & v \leftarrow v/2, r \leftarrow 2r; \\ u > v : & u \leftarrow (u - v)/2, r \leftarrow r + s, s \leftarrow 2s; \\ \text{else} : & v \leftarrow (v - u)/2, s \leftarrow s + r, r \leftarrow 2r; \end{cases}$$

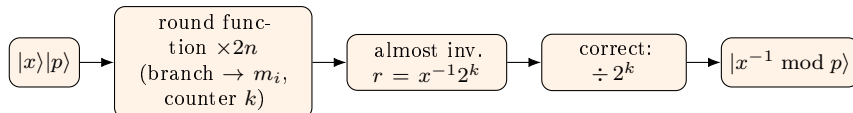
until $v = 0$; then $r \equiv x^{-1} 2^k \bmod p$ (an “almost inverse”; a final correction removes the 2^k).

Making a data-dependent loop reversible

The number of steps k (between n and $2n$) depends on the input x , so the naive loop is irreversible (different x halt at different times). Two devices fix this (Roetteler et al.):

- run a **fixed** $2n$ rounds; a flag f (initially 1) flips to 0 once $v = 0$ is reached, after which each round merely increments a **counter** k instead of touching (u, v, r, s) ;
- each round stores which of the four branches was taken in one qubit m_i (2 bits collapse to 1 by an uncomputation), so the $2n$ round-records $\{m_i\}$ let the whole thing be uncomputed after the result is copied out.

The counter needs $\lceil \log_2 n \rceil + 1$ qubits; the $2n$ round qubits m_i are the dominant ancilla cost, giving the $9n + 2\lceil \log_2 n \rceil + 10$ total of section 7.



Because inversion is $\Theta(n)$ multiplications’ worth of work and appears once per point addition, it dominates the elliptic-curve oracle – the point emphasised after theorem 2.1.

5 Reversible controlled point addition

What this section establishes

We assemble the group-law formulas of theorem 2.1 into a reversible circuit that adds a *constant* point (x_2, y_2) to a *quantum* point (x_1, y_1) , controlled on one qubit, using the blocks of section 4. This is the box $+ [2^j]P$ (or $+ [2^j]Q$) of algorithm 3.1.

5.1 The operation sequence

The point $(x_2, y_2) = [2^j]P$ is a compile-time constant, so subtractions/additions of x_2, y_2 and their multiples are *constant* operations. From theorem 2.1, with $\Delta x = x_1 - x_2$, $\Delta y = y_1 - y_2$, $\lambda = \Delta y / \Delta x$:

$$x_3 = \lambda^2 - x_1 - x_2 = \lambda^2 - \Delta x - 2x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

A reversible, garbage-free realisation (Roetteler et al.) runs:

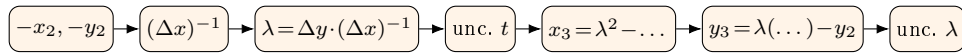
Algorithm 5.1 (Controlled point addition of a constant). Registers hold the quantum point (x_1, y_1) ; ancillas hold λ and inversion scratch; ctrl is the control qubit. Controlled on ctrl:

1. $x_1 \leftarrow x_1 - x_2, y_1 \leftarrow y_1 - y_2$ (constant modular subtractions)
2. $t \leftarrow (\Delta x)^{-1} \bmod p$ (section 4.5, modular inversion)
3. $\lambda \leftarrow \Delta y \cdot t \bmod p$; uncompute t by reverse-inversion (using Δx)
4. $x_1 \leftarrow \lambda^2 - \Delta x - 2x_2 \pmod{p}$ (square + constant sub); now $x_1 = x_3$
5. $y_1 \leftarrow \lambda(x_2 - x_3) - y_2 \pmod{p}$ (mult + constant); now $y_1 = y_3$
(uses the symmetric form $y_3 = \lambda(x_2 - x_3) - y_2$, valid since λ is the common slope)
6. uncompute λ using $\lambda = (y_3 - y_2)/(x_3 - x_2)$, recomputable from the *output* (x_3, y_3) and the constant (x_2, y_2) .

If ctrl = 0 every step is skipped and (x_1, y_1) is returned unchanged.

Why λ can be uncomputed (no garbage)

The slope is symmetric in the two summands: from the *result* (x_3, y_3) and the constant (x_2, y_2) one has $\lambda = (y_3 - y_2)/(x_3 - x_2)$ as well as $\lambda = (y_1 - y_2)/(x_1 - x_2)$. So after (x_3, y_3) is written, the same inversion-and-multiply that *computed* λ from the input recomputes it from the output and runs backward to clear the λ ancilla. This is Bennett compute-copy-uncompute specialised to the group law, and it is why the point addition leaves **zero** garbage – mandatory, since garbage would tag the branch and kill the interference of theorem 1.3.



Per point addition: *one* modular inversion, a handful of modular multiplications/squarings, and $O(1)$ constant add/subtracts – so $\Theta(n^2)$ Toffoli, dominated by the inversion. Controlling the whole block on ctrl adds a control to each gate (constant-controlled gates simplify).

5.2 Exceptional cases

Theorem 2.1 excludes $P_1 = \pm P_2$ and the point at infinity. Three remarks:

- **Doubling** ($P_1 = P_2$) uses the tangent slope $\lambda = (3x_1^2 + a)/(2y_1)$; because the added point is a *constant* $[2^j]P \neq$ the quantum point generically, doublings inside a single controlled addition do not arise.

- **Infinity** is encoded by a flag qubit (a distinguished bit pattern), and the addition is conditioned to pass the other summand through when either point is $\mathcal{O}$.
- **Collisions** $x_1 = x_2$ (giving $\Delta x = 0$, no inverse) occur only on a measure-zero fraction of inputs; Roetteler et al. show they can be avoided by the choice of curve/base-point and do not affect the resource count. The affine formulas suffice at the sizes analysed; complete (edwards/projective) formulas remove the exceptions at extra cost if desired.

6 Worked example over $\mathbb{F}_5$

We run algorithm 3.1 on the curve of section 2.4: $E : y^2 = x^3 + 3x + 2$ over $\mathbb{F}_5$, $P = (2, 1)$ of order $r = 5$, $Q = (1, 1)$, true answer $m = 3$. We use the exact form (index registers of dimension $r = 5$, QFT_5), which shows the mechanism cleanly.

Group and oracle values. From section 2.4,

$$[0]P = \mathcal{O}, [1]P = (2, 1), [2]P = (1, 4), [3]P = (1, 1) = Q, [4]P = (2, 4).$$

The oracle value is $f(k, \ell) = [k]P + [\ell]Q = [k + 3\ell \bmod 5]P$; e.g. $f(2, 4) = [2 + 12]P = [14 \bmod 5]P = [4]P = (2, 4)$.

Hidden subgroup. $L = \langle (-3, 1) \rangle = \langle (2, 1) \rangle \leq \mathbb{Z}/5\mathbb{Z} \times \mathbb{Z}/5\mathbb{Z}$, so $f(k, \ell)$ is constant exactly when $k + 3\ell$ is constant mod 5.

State evolution.

$$\frac{1}{5} \sum_{k, \ell \in \mathbb{Z}/5\mathbb{Z}} |k\rangle|\ell\rangle|\mathcal{O}\rangle \xrightarrow{f} \frac{1}{5} \sum_{k, \ell} |k\rangle|\ell\rangle|[k + 3\ell]P\rangle \xrightarrow{\text{meas. pt}} \frac{1}{\sqrt{5}} \sum_{\ell} |c - 3\ell\rangle|\ell\rangle \xrightarrow{\text{QFT}_5^{\otimes 2}} \frac{1}{\sqrt{5}} \sum_{y_1} \omega_5^{cy_1} |y_1\rangle|3y_1\rangle,$$

by theorem 1.3 with $m = 3$. Measuring gives (y_1, y_2) with $y_2 \equiv 3y_1 \pmod{5}$.

Recovering m . Inverses mod 5: $1^{-1}=1, 2^{-1}=3, 3^{-1}=2, 4^{-1}=4$. Every non-zero outcome yields $m = 3$:

outcome (y_1, y_2)	check $y_2 \equiv 3y_1$	$m = y_2 y_1^{-1} \bmod 5$	result
(1, 3)	$3 \equiv 3$	$3 \cdot 1 = 3$	$m = 3 \checkmark$
(2, 1)	$1 \equiv 6$	$1 \cdot 3 = 3$	$m = 3 \checkmark$
(3, 4)	$4 \equiv 9$	$4 \cdot 2 = 8 \equiv 3$	$m = 3 \checkmark$
(4, 2)	$2 \equiv 12$	$2 \cdot 4 = 8 \equiv 3$	$m = 3 \checkmark$
(0, 0)	—	(undefined)	retry

Since $r = 5$ is prime, a single run returns $m = 3$ unless $y_1 = 0$ (probability $1/5$). Verification: $[3]P = (1, 1) = Q$. $\checkmark$

7 Resources and comparison with RSA

What this section establishes

Concrete costs, from the Toffoli-network simulation of Roetteler, Naehrig, Svore and Lauter. The headline: at matched classical security, breaking elliptic-curve crypto takes *fewer* qubits than breaking RSA, because the key – and hence the field size n – is an order of magnitude smaller while both scale as $\Theta(n^3)$.

7.1 Asymptotic and concrete counts

For an n -bit prime field, the controlled point addition of section 5 uses

$$9n + 2\lceil \log_2 n \rceil + 10 \text{ logical qubits,} \quad \lesssim 224 n^2 \log_2 n + 2045 n^2 \text{ Toffoli gates.}$$

Shor’s ECDLP runs $2n$ controlled point additions sequentially (no extra qubits), so the whole algorithm costs

$$\approx 448 n^3 \log_2 n + 4090 n^3 \text{ Toffoli gates,} \quad \Theta(n) \text{ qubits.}$$

The inversion (section 4.5) supplies the $2n$ round-record ancillas that dominate the qubit count; the $\Theta(n^3)$ Toffoli count is dominated by the modular multiplications inside the point additions – *not* by any Fourier transform (the two QFTs are $\Theta(n^2)$, or $O(n \log n)$ approximate, and negligible).

7.2 Elliptic curves vs. RSA

	ECDLP, P-256	RSA-3072 (factoring)	ratio
classical security	≈ 128 -bit	≈ 128 -bit	matched
field / modulus size n	256	3072	$12\times$
logical qubits	2330	6146	$0.38\times$
Toffoli gates	$\approx 1.26 \times 10^{11} \dagger$	$\approx 1.86 \times 10^{13}$	

[†] point-addition core; the full run is $2n$ such additions. Figures from the LIQUi| Toffoli-network simulation of Roetteler et al. [1]; RSA figures align with Proos–Zalka [3].

Why ECC is the easier quantum target

At comparable classical security a 256-bit elliptic curve matches a 3072-bit RSA modulus, but both quantum attacks scale as $\Theta(n^3)$ in the operand size n . The $12\times$ smaller n therefore makes ECDLP dramatically cheaper in qubits (2330 vs. 6146), *despite* elliptic-curve point addition being individually harder than modular multiplication (it needs an inversion). In short: the small key that makes ECC attractive classically is exactly what makes it fragile against a quantum computer. These are logical-resource estimates for a uniformly good hardware model; physical overheads (error correction, magic-state distillation) multiply both columns similarly.

8 Summary

1. **ECDLP = 2D hidden subgroup problem.** $f(k, \ell) = [k]P + [\ell]Q = [k + m\ell]P$ hides $L = \langle (-m, 1) \rangle \leq \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ (proposition 1.2).
2. **Measured relation.** $\text{QFT}_r \otimes \text{QFT}_r$ samples a character trivial on L , giving (y_1, y_2) with $y_2 \equiv my_1 \pmod{r}$ (theorem 1.3); $m = y_2 y_1^{-1}$, one run for prime r (corollary 1.4).
3. **Oracle.** Scalar multiplication $[k]P + [\ell]Q$ is a cascade of controlled additions of the classical points $[2^j]P, [2^j]Q$ (proposition 3.3); the full circuit and the one-ancilla semiclassical circuit are in section 3.
4. **Group law.** $x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$, with λ a field *quotient* – hence a modular inversion (theorem 2.1).

5. **Reversible arithmetic.** MAJ/UMA adder (theorem 4.2) $\rightarrow$ modular add (proposition 4.3) $\rightarrow$ double (proposition 4.4) $\rightarrow$ multiply/square (theorem 4.5) $\rightarrow$ Kaliski inversion made reversible by a round counter (section 4.5).
6. **Point addition** orchestrates these garbage-free by uncomputing λ from the output (algorithm 5.1).
7. **Cost.** $9n + 2\lceil \log_2 n \rceil + 10$ qubits, $\approx 448n^3 \log_2 n + 4090n^3$ Toffoli; ECDLP-256 is cheaper in qubits than RSA-3072 (section 7).
8. **Worked example.** $\mathbb{F}_5$, $P = (2, 1)$, $Q = (1, 1) \Rightarrow m = 3$, recovered from any (y_1, y_2) with $y_1 \neq 0$ (section 6).

A Gate and cost reference

Operation	Maps	Cost (Toffoli, qubits)
integer add (CDKM)	$ a\rangle b\rangle \mapsto a\rangle a+b\rangle$	$2n$ TOF, $2n+2$ qb
modular add	$ x\rangle y\rangle \mapsto x\rangle (x+y) \bmod p\rangle$	$\Theta(n)$ TOF, $+1$ anc.
modular double	$ x\rangle \mapsto 2x \bmod p\rangle$	$\Theta(n)$ TOF, $n+2$ qb
modular multiply	$ x\rangle y\rangle 0\rangle \mapsto x\rangle y\rangle xy \bmod p\rangle$	$\Theta(n^2)$ TOF, $3n+2$ qb
modular square	$ x\rangle 0\rangle \mapsto x\rangle x^2 \bmod p\rangle$	$\Theta(n^2)$ TOF, $2n+3$ qb
modular inverse	$ x\rangle \mapsto x^{-1} \bmod p\rangle$	$\Theta(n^2)$ TOF, $O(n)$ anc.
point addition (ctrl)	$(x_1, y_1) \mapsto (x_1, y_1) + [2^j]P$	$\lesssim 224n^2 \log_2 n + 2045n^2$
full ECDLP	recover m	$\approx 448n^3 \log_2 n + 4090n^3$

References

- [1] M. Roetteler, M. Naehrig, K. M. Svore, K. Lauter, *Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms*, arXiv:1706.06752v3 (2017). Primary source for sections 3 to 5 and 7.
- [2] P. W. Shor, *Algorithms for quantum computation: discrete logarithms and factoring*, Proc. 35th FOCS, 124–134 (1994); SIAM J. Comput. **26**(5), 1484–1509 (1997).
- [3] J. Proos, C. Zalka, *Shor’s discrete logarithm quantum algorithm for elliptic curves*, Quantum Inf. Comput. **3**(4), 317–344 (2003); arXiv:quant-ph/0301141.
- [4] S. A. Cuccaro, T. G. Draper, S. A. Kutin, D. P. Moulton, *A new quantum ripple-carry addition circuit*, arXiv:quant-ph/0410184 (2004).
- [5] Y. Takahashi, S. Tani, N. Kunihiro, *Quantum addition circuits and unbounded fan-out*, Quantum Inf. Comput. **10**, 872–890 (2010).
- [6] B. S. Kaliski, *The Montgomery inverse and its applications*, IEEE Trans. Computers **44**(8), 1064–1065 (1995).
- [7] C. Gidney, *Halving the cost of quantum addition*, Quantum **2**, 74 (2018); arXiv:1709.06648.
- [8] R. B. Griffiths, C.-S. Niu, *Semiclassical Fourier transform for quantum computation*, Phys. Rev. Lett. **76**, 3228–3231 (1996).
- [9] P. Lee, *Cracking the ECDLP with Qiskit*, <https://p51lee.github.io/quantum-computing/qiskit-ecdlp/>; and Classiq docs, *Solving the ECDLP with Shor’s Algorithm*, https://docs.classiq.io/explore/algorithms/number_theory_and_cryptography/elliptic_curves/elliptic_curve_discrete_log.